

1. Introduction to Adaptive Filtering, Wiener-Hopf Equations, and Normal Equations

Iván López-Espejo

Array and Sensor Signal Processing
Aalborg University

ivl@es.aau.dk

Friday 30th July, 2021

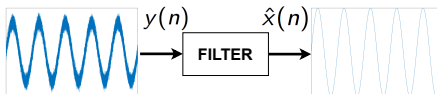
Overview

- 1 Introduction to Adaptive Filtering
- 2 Stochastic Processes: The Basics
- 3 Wiener Filtering
- 4 Wiener-Hopf Equations
- 5 Least-Squares Filtering
- 6 Normal Equations
- 7 Bibliography
- 8 Assignment

Introduction to Adaptive Filtering

- **Filter:** system that is designed to extract information about a prescribed quantity of interest from noisy data

- $y(n) = x(n) + v(n)$, where $v(n) \sim \mathcal{N}$ is noise



- Filters $f(\cdot)$ can be linear or non-linear: $\hat{x}(n) = f(y(n))$
 - *Example 1:* the (linear) Wiener filter is optimum in the mean-square-error sense
 - *Example 2:* the (linear) Kalman filter is more appropriate to deal with non-stationary signals
- **Adaptive filter:** filter relying on a *recursive algorithm* that makes it possible to perform well under non-stationary conditions (*tracking*)
 - Non-linear systems (*data dependent*)

Introduction to Adaptive Filtering

- That said, an adaptive filter $f(\cdot)$ is **linear** if it follows the *principle of superposition* whenever its parameters are fixed:

- ① $f(y_1(n) + y_2(n)) = f(y_1(n)) + f(y_2(n))$ (additivity)

- ② $f(\alpha y(n)) = \alpha f(y(n))$ (homogeneity)

- **A reminder...**

- ① $\hat{x}(n) = \sum_{i=0}^P b_i y(n-i)$ (**FIR**)

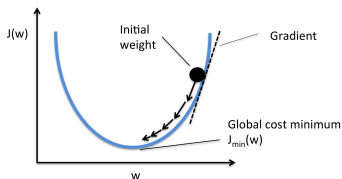
- ② $\hat{x}(n) = \frac{1}{a_0} \left(\sum_{i=0}^P b_i y(n-i) - \sum_{j=1}^Q a_j \hat{x}(n-j) \right)$ (**IIR**)

- Unlike IIR filters, FIR filters are inherently *stable* $\Rightarrow$ FIR filters as the structural basis for the design of linear adaptive filters

Introduction to Adaptive Filtering

Two main approaches to derive recursive algorithms for linear adaptive filters

- **Stochastic Gradient Descent (SGD):**

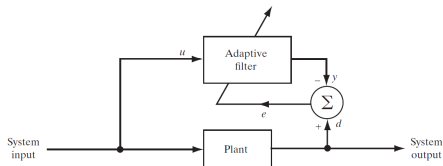


Least-mean-square (LMS) algorithm

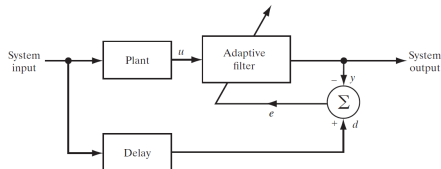
- **Least Squares:** mean squared error (MSE) minimization is carried out using algebraic matrix manipulations unlike SGD
- Recursive least-squares (RLS) algorithm
- RLS converges faster and has a higher computational complexity than LMS

Introduction to Adaptive Filtering

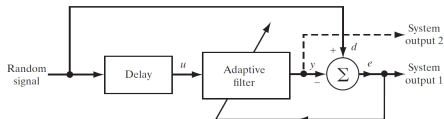
Applications



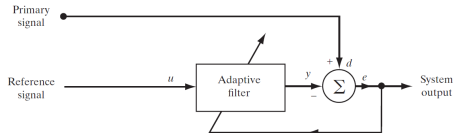
System identification



Inverse modeling (equalization)



Prediction



Noise cancellation

Stochastic Processes: The Basics

- **Stochastic process:** random process or statistical phenomenon evolving over time (e.g., speech signal)
- A stochastic process can be either *continuous* or *discrete*, e.g., $u(n)$
- **Definitions:**
 - 1 Mean: $\mu(n) = \mathbb{E}[u(n)]$
 - 2 Autocorrelation: $r(n, n - k) = \mathbb{E}[u(n)u^*(n - k)]$
 - 3 Autocovariance: $c(n, n - k) = \mathbb{E}[(u(n) - \mu(n))(u(n - k) - \mu(n - k))^*]$
 - 4 $c(n, n - k) = r(n, n - k) - \mu(n)\mu^*(n - k)$
- A stochastic process is a *wide-sense stationary* (WSS) process if...
 - 1 $\mu(n) = \mu \quad \forall n$
 - 2 $r(n, n - k) = r(k)$ and $c(n, n - k) = c(k)$

Stochastic Processes: The Basics

$$\mathbf{u}(n) = (u(n), u(n-1), \dots, u(n-M+1))^T$$

Correlation matrix:

$$\mathbf{R} = \mathbb{E} [\mathbf{u}(n)\mathbf{u}^H(n)] = \begin{bmatrix} r(0) & r(1) & \cdots & r(M-1) \\ r(-1) & r(0) & \cdots & r(M-2) \\ \vdots & \vdots & \ddots & \vdots \\ r(-M+1) & r(-M+2) & \cdots & r(0) \end{bmatrix}$$

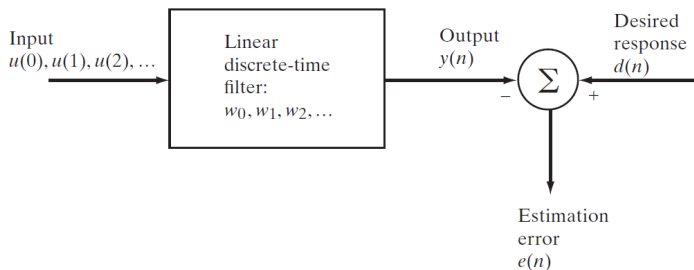
The correlation matrix is Hermitian, i.e., $\mathbf{R} = \mathbf{R}^H$, so $r(-k) = r^*(k)$:

$$\mathbf{R} = \begin{bmatrix} r(0) & r(1) & \cdots & r(M-1) \\ r^*(1) & r(0) & \cdots & r(M-2) \\ \vdots & \vdots & \ddots & \vdots \\ r^*(M-1) & r^*(M-2) & \cdots & r(0) \end{bmatrix}$$

Then, $\mathbf{R}$ is Toeplitz

Wiener Filtering

Wiener filters: a class of linear optimum discrete-time filters



$$y(n) = \sum_{k=0}^{\infty} w_k^* u(n-k)$$

- $u(n)$ and $d(n)$ are **WSS stochastic processes** with zero mean
- The goal is to design a filter minimizing the MSE
 $\mathcal{J} = \mathbb{E}[e(n)e^*(n)] = \mathbb{E}[|e(n)|^2] = \mathbb{E}[|d(n) - y(n)|^2]$

Wiener optimal solution:

$$\nabla_k \mathcal{J} = \nabla_k \mathbb{E}[e(n)e^*(n)] = -2\mathbb{E}[u(n-k)e^*(n)]$$

$$\boxed{\nabla_k \mathcal{J} = \mathbb{E}[u(n-k)e_o^*(n)] = 0}$$

- **Principle of orthogonality:** when the filter operates optimally (i.e., $\mathcal{J}$ is minimum), $e_o(n)$ is orthogonal to each input sample $u(n-k)$ contributing to the estimation of the desired response
- *Corollary to the principle of orthogonality:* $\mathbb{E}[y_o(n)e_o^*(n)] = 0$

Wiener-Hopf Equations

We can further develop the Wiener optimal solution:

$$\mathbb{E}[u(n-k)e_o^*(n)] = \mathbb{E}\left[u(n-k)\left(d^*(n) - \sum_{i=0}^{\infty} w_{oi}u^*(n-i)\right)\right] = 0$$

$$\sum_{i=0}^{\infty} w_{oi}\mathbb{E}[u(n-k)u^*(n-i)] = \mathbb{E}[u(n-k)d^*(n)]$$

Notice that...

- 1 $\mathbb{E}[u(n-k)u^*(n-i)] = r(i-k)$
- 2 $\mathbb{E}[u(n-k)d^*(n)] = p(-k)$

Wiener-Hopf equations:

$$\sum_{i=0}^{\infty} w_{oi}r(i-k) = p(-k)$$

Wiener-Hopf Equations

For a FIR filter with M weights $\Rightarrow \sum_{i=0}^{M-1} w_{oi} r(i-k) = p(-k)$

In matrix form: $\mathbf{R}\mathbf{w}_o = \mathbf{p}$

Notice that...

① $\mathbf{R} = \mathbb{E} [\mathbf{u}(n)\mathbf{u}^H(n)]$

② $\mathbf{w}_o = (w_{o0}, w_{o1}, \dots, w_{o,M-1})^\top$

③ $\mathbf{p} = \mathbb{E} [\mathbf{u}(n)d^*(n)] = (p(0), p(-1), \dots, p(1-M))^\top$

Then, the optimal filter weights are simply calculated as

$$\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$$

Least-Squares Filtering

- Wiener filtering (*ensemble averages*, WSS) *versus* least squares (*batch-processing approach*)
- Multiple linear regression model:

$$d(i) = \sum_{k=0}^{M-1} w_{ok}^* u(i-k) + e_o(i) \quad (e_o(i) \text{ is the measurement error})$$

- **Linear least-squares filter:** set of weights minimizing

$$\sum_{i=i_1}^{i_2} |e(i)|^2$$

- The **covariance method** makes no assumptions about the data ($u(1), u(2), \dots, u(N)$) outside the interval $[1, N]$, i.e., $i_1 = M$ and $i_2 = N$

Least-Squares Filtering

As for Wiener filtering...

$$\mathcal{E} = \sum_{i=M}^N e(i)e^*(i) = \sum_{i=M}^N |e(i)|^2$$

$$\nabla_k \mathcal{E} = 0$$

$$\sum_{i=M}^N u(i-k)e_{\min}^*(i) = 0$$

- **Principle of orthogonality:** when the FIR filter operates optimally (i.e., in its least-squares condition), the minimum-error time series $e_{\min}(i)$ is orthogonal to the time series $u(i-k)$

- *Corollary to the principle of orthogonality:* $\sum_{i=M}^N y_{\min}(i)e_{\min}^*(i) = 0$

Normal Equations

Normal equations: an alternative to the principle of orthogonality to describe the least-squares condition of a FIR filter

$$\sum_{i=M}^N u(i-k)e_{\min}^*(i) = 0 \text{ and } e_{\min}(i) = d(i) - \sum_{t=0}^{M-1} \hat{w}_t^* u(i-t)$$
$$\sum_{t=0}^{M-1} \hat{w}_t \sum_{i=M}^N u(i-k)u^*(i-t) = \sum_{i=M}^N u(i-k)d^*(i)$$

Notice that...

$$\textcircled{1} \sum_{i=M}^N u(i-k)u^*(i-t) = \phi(t, k)$$

$$\textcircled{2} \sum_{i=M}^N u(i-k)d^*(i) = z(-k)$$

Normal equations:

$$\sum_{t=0}^{M-1} \hat{w}_t \phi(t, k) = z(-k)$$

Normal Equations

Normal equations in matrix form

$$\Phi = \begin{bmatrix} \phi(0,0) & \phi(1,0) & \cdots & \phi(M-1,0) \\ \phi(0,1) & \phi(1,1) & \cdots & \phi(M-1,1) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(0,M-1) & \phi(1,M-1) & \cdots & \phi(M-1,M-1) \end{bmatrix}$$

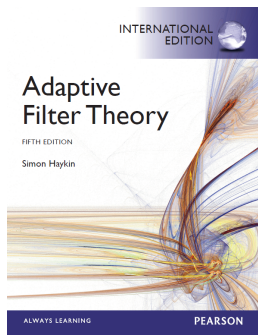
$$\mathbf{z} = (z(0), z(-1), \dots, z(-M+1))^{\top}$$

$$\hat{\mathbf{w}} = (\hat{w}_0, \hat{w}_1, \dots, \hat{w}_{M-1})^{\top}$$

Normal equations: $\Phi \hat{\mathbf{w}} = \mathbf{z}$

Then, the optimal set of weights can be calculated as

$$\hat{\mathbf{w}} = \Phi^{-1} \mathbf{z}$$



Simon Haykin, “Adaptive Filter Theory (5th Edition)”. Pearson, 2014

- *Introduction to Adaptive Filtering: Background and Preview*
- *Stochastic Processes: The Basics: 1.1 and 1.3*
- *Wiener Filtering: 2.1 and 2.2*
- *Wiener-Hopf Equations: 2.4*
- *Least-Squares Filtering: 9.1, 9.2 and 9.3*
- *Normal Equations: 9.5*

Assignment: Wiener Filtering

Consider the autoregressive process described by the difference equation

$$d(n) = 0.75d(n-1) + v(n),$$

and the noisy process

$$u(n) = d(n) + w(n) + 0.5w(n-1),$$

where both $v(n)$ and $w(n)$ are white noise with zero mean and unit standard deviation, and $\mathbb{E}[v(n)w(n)] = 0$. We are interested in a **first-order Wiener filter** estimating $d(n)$ from $u(n)$.

- 1 Draw the block diagram of the system
- 2 Write the error signal $e(n)$ and the cost function to be minimized
- 3 Calculate, **by hand**, expressions for the optimal set of weights as well as obtain the values of the optimal weights (**explain, step by step, how you do it!**)

Assignment: Wiener Filtering

- 4 Obtain a numerical approximation of the optimal weights. For that, using, e.g., MatLab or Python, generate 10,000 samples of $d(n)$ and $u(n)$, where $d(0) = w(0) = 0$, and apply the expressions calculated in 3). Compare the result with that from 3)
- 5 Using the estimated Wiener filter, apply it to $u(n)$ to calculate the estimate $\hat{d}(n)$. **Both by hand and using numerical approximation**, calculate the error variances $\sigma_{du}^2 = \mathbb{E} [(d(n) - u(n))^2]$ and $\sigma_{d\hat{d}}^2 = \mathbb{E} \left[\left(d(n) - \hat{d}(n) \right)^2 \right]$ and compare them (**when by hand, explain, step by step, how you do it!**)

1. Introduction to Adaptive Filtering, Wiener-Hopf Equations, and Normal Equations

Iván López-Espejo

Array and Sensor Signal Processing
Aalborg University

ivl@es.aau.dk

Friday 30th July, 2021